DevOps Lab Exam: Quick Run Guide

Exps 1, 2, 3, 4, 6 and 7. Every command is copy-paste ready for an Ubuntu terminal (bash).

Before running anything, replace **YOURUSER** with your Docker Hub username and **YOURGITHUB** with your GitHub username.

Before you start (5 minutes)

Check your tools:

```
git --version
docker --version
docker ps
kubectl version --client
minikube version
```

Have these ready:

- **GitHub token** for pushing over HTTPS: GitHub > Settings > Developer settings > Personal access tokens > generate with the **repo** scope. Paste it when Git asks for a password.
- **Docker Hub access token**: Docker Hub > Account settings > Personal access tokens > generate with Read & Write. Use it as the password for `docker login` and as the `DOCKERHUB_TOKEN` secret.
- Git identity (once):

```
git config --global user.name 'Your Name'
git config --global user.email 'you@example.com'
```

Suggested time plan (about 55 minutes):

Exp	Topic	Time	Slow part
1	Git push, branch, merge	5 min	none
2	Flask app, Docker Hub	8 min	image build and push
3	GitHub Actions CI to Docker Hub	10 min	waiting for the Actions run
4	Jenkins in Docker	12 min	plugin install
6	kubectl basics on Minikube	10 min	cluster start
7	Python app on Kubernetes	8 min	image pull

Exp 1: Git Essentials (commit, push, merge, pull requests)

Idea: create a local repository, commit a file, and push it to an empty GitHub repository.

First, on GitHub: create a new **empty** repository named `git-essentials` (no README, no .gitignore).

Steps

```
mkdir git-essentials
cd git-essentials
echo 'Git Essentials Experiment' > README.md

git init
git add .
git commit -m 'Initial commit'

git branch -M main
git remote add origin https://github.com/YOURGITHUB/git-essentials.git
git push -u origin main
```

At the password prompt, paste your **GitHub token**. Refresh the repository page to see the file.

Branch, merge and pull (if the exam asks)

```
git status
git checkout -b feature/new-feature
echo 'New feature' > feature.txt
git add .
git commit -m 'Add feature'
git checkout main
git merge feature/new-feature
git push origin main
git pull origin main
git log --oneline
```

Pull request (if asked)

```
git checkout -b feature/pr-demo
echo 'PR demo' > pr.txt
git add .
git commit -m 'Add PR demo'
git push -u origin feature/pr-demo
```

On GitHub, click **Compare & pull request**, then **Create pull request**, then **Merge pull request**. Back in the terminal: `git checkout main` and `git pull origin main`.

Change repository visibility (if asked)

Repository > **Settings** > scroll to **Danger Zone** > **Change visibility** > choose Private or Public > type the repository name > **I understand, change repository visibility**.

If it fails

- **`Authentication failed`**: you typed your GitHub password. Use the token instead.
- **`remote origin already exists`**: run `git remote set-url origin https://github.com/YOURGITHUB/git-essentials.git`.
- **`failed to push, rejected`**: the GitHub repository was not empty. Run `git pull origin main --allow-unrelated-histories`, then push again.

Exp 2: Flask App Containerization and Docker Hub Deployment

Idea: write a small Flask app, package it as a Docker image, run it, and push the image to Docker Hub.

Create the files

```
mkdir flask-docker-app && cd flask-docker-app

cat > app.py << 'EOF'
from flask import Flask, render_template_string
app = Flask(__name__)

HTML = '''<!DOCTYPE html>
<html lang='en'>
<head><meta charset='UTF-8'><title>Flask Docker App</title>
<style>
body {background: linear-gradient(to right, #00c6ff, #0072ff); color: white;
font-family: Arial, sans-serif; text-align: center; padding-top: 100px;}
.card {background: rgba(255,255,255,0.1); padding: 2em; border-radius: 20px;
width: 50%; margin: auto;}
</style></head>
<body><div class='card'>
<h1>Welcome to Flask + Docker!</h1>
<p>This web page is running inside a Docker container.</p>
</div></body></html>'''
```

```
@app.route('/')
def home():
    return render_template_string(HTML)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
EOF

cat > requirements.txt << 'EOF'
flask==3.0.0
EOF

cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
EOF
```

Build and run

```
docker build -t YOURUSER/flask-docker-app:latest .
docker run -d --name flask-app -p 5000:5000 YOURUSER/flask-docker-app:latest
docker ps
curl http://localhost:5000
```

Open `http://localhost:5000` in the browser. You should see the blue welcome page.

Push to Docker Hub

```
docker login -u YOURUSER
docker push YOURUSER/flask-docker-app:latest
```

The password is your **Docker Hub access token**. Verify at `https://hub.docker.com/r/YOURUSER/flask-docker-app`.

Clean up

```
docker stop flask-app && docker rm flask-app
```

If it fails

- **Port 5000 in use:** run with `-p 5001:5000` and open `localhost:5001`.
- **`denied: requested access to the resource is denied` on push:** the image name must start with your Docker Hub username, and you must be logged in.
- **`permission denied` on the Docker socket:** run `sudo usermod -aG docker $USER && newgrp docker`.

Exp 3: CI Pipeline (Python app, GitHub Actions, Docker Hub)

Idea: on every push to `main`, GitHub Actions runs the tests, builds a Docker image and pushes it to Docker Hub.

First, on GitHub: create an **empty** repository named `python-ci-docker-lab`. Then add the two secrets (Settings > Secrets and variables > Actions > New repository secret) **before the first push**:

- `DOCKERHUB_USERNAME`: your Docker Hub username
- `DOCKERHUB_TOKEN`: your Docker Hub access token

Create all the files

```
mkdir python-ci-docker-lab && cd python-ci-docker-lab
mkdir -p tests .github/workflows

cat > app.py << 'EOF'
def add(a, b):
    return a + b

if __name__ == '__main__':
    print('Hello from Python CI Lab!')
    print('2 + 3 =', add(2, 3))
EOF

cat > tests/test_app.py << 'EOF'
from app import add

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0
EOF

touch conftest.py
```

```
cat > requirements.txt << 'EOF'
pytest==8.3.2
EOF

cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
EOF

cat > .gitignore << 'EOF'
__pycache__/
.venv/
.pytest_cache/
.DS_Store
*.pyc
EOF

cat > .github/workflows/ci-dockerhub.yml << 'EOF'
name: ci-dockerhub

on:
  push:
    branches: [ main ]
    tags: [ '*' ]
  pull_request:
    branches: [ main ]

jobs:
  build-test-push:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install deps
        run: |
          python -m pip install --upgrade pip
```

```
          pip install -r requirements.txt

      - name: Run tests
        run: pytest -q

      - name: Docker meta
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ secrets.DOCKERHUB_USERNAME }}/python-ci-lab
          tags: |
            type=raw,value=latest,enable={{is_default_branch}}
            type=sha,prefix=sha-,format=short
            type=ref,event=tag

      - name: Set up QEMU
        uses: docker/setup-qemu-action@v3

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and push
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          platforms: linux/amd64
EOF
```

Test locally (optional, 1 minute)

```
pip install -r requirements.txt
python app.py
pytest -q
```

Push to GitHub

```
git init
git add .
git commit -m 'init lab'
git branch -M main
git remote add origin https://github.com/YOURGITHUB/python-ci-docker-lab.git
git push -u origin main
```

Watch the pipeline and verify

1. GitHub > **Actions** tab > open the latest run. Wait for the green tick (1 to 2 minutes). Click **Run tests** and **Build and push** to show their logs.
2. Open `https://hub.docker.com/r/YOURUSER/python-ci-lab` > **Tags**. You should see `latest` and `sha-xxxxxxx`.

Release with a tag

```
git tag v1.0.0
git push origin v1.0.0
```

Wait for the new Actions run, then refresh the Docker Hub Tags tab. It now also shows `v1.0.0`.

If it fails

- **Failed at Login to Docker Hub:** the secret names are case-sensitive, or the token is wrong. Fix them, then click **Re-run all jobs**.
- **Failed at Run tests (`No module named app`):** `conftest.py` is missing from the project root.
- **Workflow never starts:** the file must be exactly `.github/workflows/ci-dockerhub.yml` and the branch must be `main`.
- **YAML error:** indentation matters. Use the heredoc above rather than typing by hand.

Exp 4: Jenkins Deployment Using Docker (static site pipeline)

Idea: run Jenkins as a container. A pipeline script generates an HTML site, publishes it inside Jenkins with the HTML Publisher plugin, and archives it.

Start Jenkins

```
mkdir ~/jenkins-static-site && cd ~/jenkins-static-site

cat > docker-compose.yml << 'EOF'
version: '3.8'
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: unless-stopped
    ports:
      - '8080:8080'
      - '50000:50000'
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock

volumes:
  jenkins_home:
    driver: local
EOF

docker compose up -d
```

If you get an error about mounting `/var/run/docker.sock`, delete that one line from the file. This lab does not use it.

Wait about a minute, then get the admin password:

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Set up Jenkins (in the browser)

1. Open `http://localhost:8080`. Paste the password.
2. Click **Install suggested plugins** and wait 3 to 5 minutes (this is the slow part). Create the admin user.
3. **Manage Jenkins > Plugins > Available plugins**, search **HTML Publisher**, tick it and click **Install**. Restart Jenkins if asked, then log in again.

Create the pipeline

1. **New Item** > name `karunya-site-pipeline` > **Pipeline** > OK.
2. Under **Pipeline > Definition**, choose **Pipeline script** and paste:

```
pipeline {
    agent any
    stages {
        stage('Generate site') {
            steps {
                script {
                    sh 'rm -rf site || true'
                    sh 'mkdir -p site/assets'
                    writeFile file: 'site/index.html', text: """
<!doctype html>
<html lang='en'>
<head>
  <meta charset='utf-8'/>
  <meta name='viewport' content='width=device-width, initial-scale=1'/>
  <title>Karunya University - Simple Site</title>
  <link rel='stylesheet' href='assets/style.css'/>
</head>
<body>
  <header class='site-header'>
    <div class='container'>
      <h1>Karunya University</h1>
      <p class='tagline'>Values - Knowledge - Service</p>
    </div>
  </header>
  <main class='container'>
    <section class='card'>
      <h2>About Karunya University</h2>
      <p>Karunya Institute of Technology and Sciences is a leading centre for
higher education in India.</p>
    </section>
  </main>
  <footer class='site-footer container'>
    <p>&copy; ${new Date().format('yyyy')} Karunya University - Generated by
Jenkins</p>
  </footer>
</body>
</html>
"""
                    writeFile file: 'site/assets/style.css', text: """
body { font-family: Arial, sans-serif; margin:0; }
.container { max-width:800px; margin:0 auto; padding:20px; }
.site-header { background:#0b3d91; color:#fff; padding:20px; }
.tagline { font-size:14px; opacity:0.8; }
.card { background:#fff; border:1px solid #ddd; padding:15px; margin:15px 0;
border-radius:6px; }
.site-footer { text-align:center; font-size:12px; color:#555; margin-
```

```
top:20px; }
"""
                }
            }
        }
        stage('Publish site') {
            steps {
                publishHTML([
                    allowMissing: false,
                    alwaysLinkToLastBuild: true,
                    keepAll: true,
                    reportDir: 'site',
                    reportFiles: 'index.html',
                    reportName: 'Karunya University - Simple Site'
                ])
            }
        }
    }
    post {
        always {
            archiveArtifacts artifacts: 'site/**', fingerprint: true
        }
    }
}
```

3. Click **Save**, then **Build Now**.
4. Open the build, and click **Karunya University - Simple Site** in the left menu to see the page. Archived files are listed under **Build Artifacts**.

If it fails

- **Site shows with no colours:** Jenkins blocks CSS by default. Go to **Manage Jenkins > Script Console**, run this line, then rebuild and refresh:

```
System.setProperty('hudson.model.DirectoryBrowserSupport.CSP', '')
```

- `publishHTML` **not found:** the plugin is not installed, or Jenkins was not restarted.
- **Port 8080 in use:** change `'8080:8080'` to `'9090:8080'` and use `localhost:9090`.
- **Password command prints nothing:** Jenkins is still starting. Wait a minute.

Exp 6: Manage Kubernetes Resources Using kubectl

Idea: a Pod is one running container. A Deployment keeps several copies running and handles updates. A Service gives them a stable address.

Start the cluster

```
minikube start --driver=docker
minikube status
kubectl get nodes
```

The node must show **Ready**. The first start takes 3 to 5 minutes.

Step 1: Pod

```
mkdir -p ~/k8s-lab && cd ~/k8s-lab

cat > nginx-pod.yaml << 'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:latest
      ports:
        - containerPort: 80
EOF

kubectl apply -f nginx-pod.yaml
kubectl wait --for=condition=Ready pod/nginx-pod --timeout=120s
kubectl get pods
kubectl describe pod nginx-pod
```

Open the app: run this in a **second terminal** and browse to `http://localhost:8080`:

```
kubectl port-forward pod/nginx-pod 8080:80
```

Press Ctrl+C when done, then delete the pod:

```
kubectl delete pod nginx-pod
```

Step 2: Deployment, scaling, update, rollback

```
kubectl create deployment my-nginx --image=nginx
kubectl rollout status deployment/my-nginx
kubectl get deployments
kubectl get pods -l app=my-nginx

kubectl scale deployment my-nginx --replicas=3
kubectl get pods

kubectl set image deployment/my-nginx nginx=nginx:1.25
kubectl rollout status deployment/my-nginx

kubectl rollout undo deployment/my-nginx
kubectl rollout status deployment/my-nginx
```

In `set image`, the name before `=` is the **container** name (nginx), not the deployment name.

Step 3: Expose as a Service

```
kubectl expose deployment my-nginx --type=NodePort --port=80
kubectl get svc
minikube service my-nginx --url
```

Open the printed URL in the browser. The command keeps running while you browse. Press Ctrl+C afterwards.

Step 4: Clean up

```
kubectl delete svc my-nginx
kubectl delete deployment my-nginx
kubectl get all
minikube stop
```

If it fails

- **`minikube start` fails:** make sure Docker is running (`docker ps`). Then run `minikube delete` and `minikube start --driver=docker`.
- **Pod stuck in `ContainerCreating` or `ImagePullBackOff`:** the image is still downloading. Wait a minute, then run `kubectl get pods` again.
- **Port-forward: port 8080 in use:** use `8081:80` and open `localhost:8081`.

Exp 7: Kubernetes Deployment and Service for a Python App from Docker Hub

Idea: the same Deployment and Service as Exp 6, written as YAML files and pointing at a Python (Flask) image from Docker Hub.

Start the cluster and test the image

```
minikube start --driver=docker
kubectl get nodes
docker pull dockersamples/flaskapp
```

If the pull fails with "repository does not exist", use this image instead (also port 5000), and use its name in the Deployment below:

```
docker pull thetechdarts/sampleflaskapp
```

Deployment

```
mkdir -p ~/k8s-python-lab && cd ~/k8s-python-lab

cat > python-deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: python-web-deployment
  labels:
    app: python-web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: python-web
  template:
    metadata:
      labels:
        app: python-web
    spec:
      containers:
        - name: python-web
          image: dockersamples/flaskapp:latest
          ports:
            - containerPort: 5000
EOF
```

```
kubectl apply -f python-deployment.yaml
kubectl rollout status deployment/python-web-deployment
kubectl get deployments
kubectl get pods
kubectl describe deployment python-web-deployment
```

If you use the fallback image, run this before applying:

```
sed -i 's#dockersamples/flaskapp:latest#thetechdarts/sampleflaskapp:latest#'
python-deployment.yaml
```

Service

```
cat > python-service.yaml << 'EOF'
apiVersion: v1
kind: Service
metadata:
  name: python-web-service
spec:
  type: NodePort
  selector:
    app: python-web
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
EOF

kubectl apply -f python-service.yaml
kubectl get svc
minikube service python-web-service --url
```

Open the printed URL in the browser. The command keeps running while you browse.

Clean up

```
kubectl delete -f python-service.yaml
kubectl delete -f python-deployment.yaml
minikube stop
```

If it fails

- **`ImagePullBackOff`**: the image name is wrong or does not exist. Run `kubectl describe pod <pod-name>` to see why, then use the fallback image.

- **Pods Running but the page will not load:** the labels must match in three places: the Deployment's `labels`, its `selector.matchLabels`, and the Service's `selector` (all `app: python-web`). The ports must match what the app listens on (5000). Check with `kubectl logs deploy/python-web-deployment`.
- **URL times out:** wait 20 seconds after the pods are Ready. Otherwise run `kubectl port-forward svc/python-web-service 5000:5000` and open `localhost:5000`.

Quick memory aids

- **YAML skeleton for every Kubernetes object:** `apiVersion`, `kind`, `metadata` (name, labels), `spec`.
- **Pod:** `apiVersion: v1`. **Deployment:** `apiVersion: apps/v1`. **Service:** `apiVersion: v1`.
- **Dockerfile order:** FROM, WORKDIR, COPY requirements, RUN pip install, COPY everything, CMD.
- **Git push order:** init, add, commit, branch -M main, remote add origin, push -u origin main.
- **GitHub Actions step order:** checkout, setup-python, install, pytest, docker meta, buildx, login, build and push.